SVD for NLP: Latent Semantic Analysis using Singular Value Decomposition

Massimiliano Carli
Politecnico di Torino
Torino, Italy
s337728@studenti.polito.it

Rosario Interlandi
Politecnico di Torino
Torino, Italy
s334202@studenti.polito.it

***Abstract*—This report explores the application of Singular Value Decomposition for dimensionality reduction in Natural Language Processing for Latent Semantic Analysis. The study outlines how common patterns can be found by transforming textual data into a numerical form using the TF-IDF method, followed by dimensionality reduction with Truncated SVD. We will demonstrate the effectiveness of SVD using the 20 Newsgroups dataset, visualizing the results and commenting their interpretation.**

***Index Terms*—NLP, SVD, LSA**

I. Introduction

The rapid growth of textual data in the digital age demands advanced techniques to analyze, summarize, and extract meaningful patterns from high-dimensional textual datasets, this need reflects on Natural Language Processing (NLP), which is a sub-field of data analysis that tries to carry out exactly this objective. Singular Value Decomposition (SVD) is a mathematical method widely used to reduce data dimensionality. In the context of NLP, SVD provides the extraction of the most relevant features or keywords by identifying latent semantic structures within text data. Latent semantic structure refers to the hidden relationships and patterns among a set of documents and words used in them, that are not directly observable in the original data, such as common themes or topics. The dimensionality reduction feature of SVD can be exploited for this goal, uncovering these relationships, providing a so-called Latent Semantic Analysis (LSA).

II. Methodology

A. Document-Term Matrix and preprocessing

The Document-Term Matrix (DTM) is a numerical representation of textual data. A DTM is a two-dimensional matrix where each row corresponds to a document, and each column corresponds to a unique term (word) in the dataset. The value at the intersection of a row and a column quantifies the relationship between the document and the term. Generally $DTM \in \mathbb{R}^{N \times m}$ with N total number of documents, and m number of unique terms appearing in the whole dataset.

To construct the DTM, raw text data follows some preprocessing steps:

1) **Tokenization:** Breaking down each document into individual words, called tokens or terms;
2) **Lowercasing:** Converting all text to lowercase to ensure case-insensitive coherence;
3) **Stop word removal:** Excluding common words (e.g., "and", "the", "is") that do not contribute to the meaning of sentences;
4) **Punctuation removal:** Eliminating non-alphanumeric characters that, exactly like stop words, are not meaningful for semantic analysis[1].

B. TF-IDF Matrix Computation

After preprocessing, term weights are assigned using the *Term Frequency-Inverse Document Frequency (TF-IDF)* method. Producing a DTM, named TF-IDF matrix, representing our dataset. TF-IDF captures the importance of a term by balancing its frequency in a document (Term Frequency - TF) against how common the term is across all documents (Inverse Document Frequency - IDF). Formally

$$\text{TF}(t,d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

$$\text{IDF}(t) = \log\left(\frac{N}{n_t + 1}\right)$$

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \cdot \text{IDF}(t) = \text{TF}(t,d) \cdot \log\left(\frac{N}{n_t + 1}\right)$$

Where:

- t: Term;
- d: Document;
- $f_{t,d}$: Absolute frequency of a term t into a document d
- n_t: Number of documents in which t appears at least one time (i.e. for which $f_{t,d} \neq 0$).

We added the one 1 to the IDF denominator for avoid the division by zero, i.e. terms that occur in all documents in a training set, will not be entirely ignored

Note that the TF-IDF definition provided differs from the standard textbook notation,

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \cdot \log\left(\frac{N}{1 + \text{DF}(t)}\right)$$

[1]Further improvements on this work could include a *Stemming* or *Lemmatization* of terms during preprocessing (i.e. reducing words to their base or root forms to group similar terms together - e.g., "cooking" becomes "cook")

we preferred to adopt the definition provided by the popular python library `scikit-learn`.

Other definitions of IDF could be adopted based on use cases, for instance smooth IDF is defined as

$$IDF(t) = \log\left(\frac{1+N}{1+\text{DF}(t)}\right) + 1$$

C. Singular Value Decomposition

Singular Value Decomposition (SVD) is a mathematical method used to decompose a matrix A into three components:

$$A = U\Sigma V^T$$

Where:

- U: A matrix containing the left singular vectors.
- Σ: A diagonal matrix with singular values.
- V^T: A matrix containing the right singular vectors.

A variation based on a randomized SVD solver [1] can be applied to reduce the dimensionality of any large dimensional matrix A by retaining only the top M components. This involves selecting the top M largest singular values from Σ, ordered in decreasing magnitude. Truncating U and V to retain only the corresponding M singular vectors, obtaining respectively U_M and V_M.

D. Right Singular Vectors

In LSA is the right singular vectors matrix $V_M^T \in \mathbb{R}^{M\times m}$ being mainly considered.

V_M^T is a transformed representation of the original TD-IDF matrix in the right singular vectors' M-reduced-space[2]. V_M^T is therefore composed of M rows - each associated to a *latent semantic factor*, which can be interpreted as the M most relevant "topics" of our dataset, and m columns, each corresponding to a unique term of the dataset. Each value in the matrix describes the relevance of a specific term with a specific topic.

For a topic i, represented by the i-th row of the V_M^T, it is possible to sort its entries based on their values, in descending order of magnitude. Following this sorting, the i-th row can be further truncated to the first p terms, retaining only the most important p terms for each topic[3].

E. Left Singular Vectors

In LSA, also the left singular vectors matrix $U_M \in \mathbb{R}^{N\times M}$ is often analyzed to unveil underlying patterns. In this case, each row of U_M represents a document, described by M entries. Similar to the interpretation we adopted for V_M^T, each entry of the matrix now represent the relationship within each document and each one of the M topics extracted by SVD.

The matrices U_M and V_M represent the relationship between documents, terms, and topics, but they do not capture the varying importance of these relationships within the latent semantic space. This varying importance is instead represented by the M singular values in Σ_M. Therefore, the products $\Sigma_M V_M^T$ and $U_M \Sigma_M$ are used in the actual implementation of LSA, rather than just the matrices U_M and V_M.

[2]Similarly to the eigenspace spanned by the Laplacian's M smallest eigenvectors in spectral clustering

[3]Note that the sorting, together with the p truncation, is done specifically for the i-th topic, it is not a global sorting of columns.

III. Results

A. Dataset Preparation

We used the 20 Newsgroups dataset, a collection of approximately 20,000 documents representing newsgroups. The data is organized into 20 different newsgroups, each corresponding to a different topic, with a well-balanced cardinality of documents for each group. Some of the newsgroups are very closely related to each other (e.g. `comp.sys.ibm.pc.hardware / comp.sys.mac.hardware`, where "comp" stands for "computer"), while others are highly unrelated (e.g `misc.forsale / soc.religion.christian`). Such dataset is provided with some useful metadata by default, we opted to remove it to focus exclusively on text content.

B. TF-IDF and SVD Application

We started by cleaning data following what described in Section II-A: removing noise-terms such stop words and punctuation. We then proceeded to build the DTM and TF-IDF matrix. DTM and TF-IDF were described by 13401 unique terms (columns) and 18846 unique documents (rows).

After applying Truncated SVD on the TF-IDF matrix, we retained $M = 8$ components, obtaining the reduced U_M matrix with size (18846×8), and the reduced V_M^T matrix of size (8×134101).

C. Visualization of $U_M\Sigma_M$ and $\Sigma_M V_M^T$

The $U_M\Sigma_M$ matrix can be visualized in a 2D scatter plot, examples of which are shown in Figure 1, displaying only two SVD columns per time for graphical purposes.

For what concern $\Sigma_M V_M^T$, we retained only the $p = 5$ top-terms for each of the 5 topics extracted to improve the interpretation of each. The final result can be observed in the following

```
Top words for topic 1:
know people like just don

Top words for topic 2:
think believe jesus people god

Top words for topic 3:
does bible windows jesus god

Top words for topic 4:
ide game god scsi drive

Top words for topic 5:
government chip key scsi drive

Top words for topic 6:
problem file drive dos windows
```

```
Top words for topic 7:
just does don know thanks

Top words for topic 8:
does chip know game key
```

We notice how the final result effectively reflects the description we provided to the 20 newsgroup dataset. The 8 topics describe a spectrum of terms between "religion" and "comp", which are intuitively the most unrelated groups of our dataset. In fact, the first topics reported are described by mainly terms about "religion", then gradually converging to terms mainly related to "comp" in the last topics; according to this interpretation the middle topics must be a mix of both "religion" and "comp": this is indeed proven observing topic 4, in which words like `ide, game, drive` got mixed with `god`, a term related to religion.

IV. Conclusions

By reducing the dimensionality of textual data, SVD uncovered latent semantic structures that are otherwise difficult to identify. We demonstrated the effectiveness of SVD for LSA using the 20 Newsgroups dataset. Through a combination of preprocessing, TF-IDF transformation, and Truncated SVD, we extracted meaningful topics and textual patterns from the dataset. Our analysis showed that the right singular vectors matrix (V_M^T), identify the most relevant terms associated with different topics. Similarly, the left singular vectors (U_M) provided insights into the relationships between documents.

References

[1] N. Halko, P.-G. Martinsson, and J. A. Tropp, "Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions," *SIAM review*, vol. 53, no. 2, pp. 217–288, 2011.

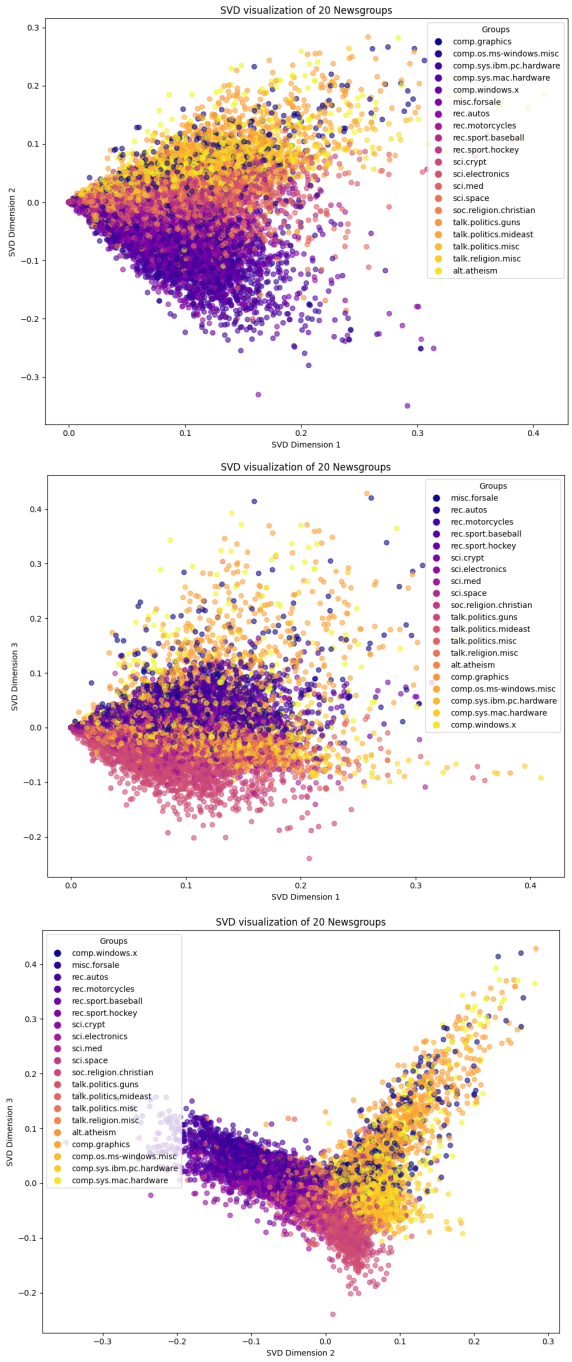

Fig. 1. Visualization of $U_M \Sigma_M$ columns as dimensions in a 2D plane. The documents, represented as points, were color-coded by their category labels provided by the dataset itself (ground-truth). We can see how SVD correctly extracted patterns from documents, collocating similar topics in geometrically near positions on the left-singular-space defined by U_M.